

BÁO CÁO MÔN HỌC

Chủ đề: Thuật toán tìm đường (Pathfinding) trong thiết kế Game: Từ Cấu trúc dữ liệu đến Tối ưu hóa không gian động

1. Mở đầu và Phát biểu bài toán

Trong kỹ thuật phát triển trò chơi điện tử hiện đại, Level Design (thiết kế màn chơi) không đơn thuần là việc sắp đặt các khối hình học hay ánh sáng mang tính nghệ thuật. Đằng sau một không gian ảo là những bài toán khoa học máy tính phức tạp, trong đó cốt lõi là **Thuật toán tìm đường (Pathfinding)**.

Bài toán đặt ra là: Làm thế nào để các thực thể nhân tạo (NPC - Non-Player Character) di chuyển từ điểm A đến điểm B một cách tự nhiên, tránh được chướng ngại vật, nhưng vẫn phải đảm bảo tốc độ tính toán trong thời gian thực (real-time)? Sự đánh đổi (trade-off) kinh điển ở đây luôn nằm giữa **độ chính xác của quỹ đạo** và **giới hạn của tài nguyên hệ thống (RAM/CPU)**. Nếu thuật toán quá cồng kềnh, trò chơi sẽ sụt giảm số khung hình trên giây (FPS); ngược lại, nếu thuật toán quá đơn giản, AI sẽ di chuyển mất tự nhiên hoặc mắc kẹt.

2. Nền tảng thuật toán và Tối ưu hóa cấu trúc dữ liệu

2.1. Phân tích thuật toán A* và hàm Heuristic

Nhắc đến Pathfinding, A* (A-Star) luôn là tiêu chuẩn vàng. Được phát triển từ nguyên lý của thuật toán Dijkstra, A* tối ưu hóa không gian tìm kiếm bằng cách đưa thêm hàm ước lượng (Heuristic) vào công thức tính chi phí:

$$f(n)=g(n)+h(n)$$

Trong đó:

- $g(n)$: Chi phí thực tế đi từ node bắt đầu đến node n.
- $h(n)$: Chi phí ước lượng (Heuristic) từ node n đến đích.

Tuy nhiên, sự xuất sắc của hệ thống không nằm ở công thức tĩnh, mà nằm ở cách lập trình viên lựa chọn hàm $h(n)$ phù hợp với đặc thù hình học của đồ thị. Ví dụ, hàm **Manhattan** phù hợp với đồ thị lưới 4 hướng di chuyển, trong khi **Chebyshev** hoặc **Euclidean** lại cần thiết cho lưới 8 hướng. Việc tính toán sai lệch Heuristic

(overestimating) có thể phá vỡ tính tối ưu của đường đi, biến A* thành thuật toán tham lam (Greedy Search).

2.2. Tối ưu hóa bằng Cấu trúc dữ liệu

Trên phương diện lập trình hướng đối tượng (OOP) bằng C++ hoặc Java, hiệu năng của thuật toán A* phụ thuộc sống còn vào cách quản lý **Open List** (danh sách các node đang chờ được duyệt). Nếu sử dụng cấu trúc Mảng (Array) thông thường, thời gian để tìm kiếm node có giá trị $f(n)$ nhỏ nhất sẽ tiêu tốn $O(N)$.

Để giải quyết vấn đề này, kiến trúc hệ thống bắt buộc phải áp dụng **Hàng đợi ưu tiên (Priority Queue)**, thường được triển khai dưới dạng **Cây Đống Cực Tiểu (Min-Heap)**. Cấu trúc này giúp giảm thời gian lấy node có trọng số nhỏ nhất xuống $O(1)$ và thời gian chèn/cập nhật node xuống mức $O(\log V)$. Nhờ vậy, độ phức tạp thời gian tổng thể của thuật toán được ép xuống mức tối ưu $O(E \log V)$ (với E là số cạnh và V là số đỉnh), đảm bảo AI có thể tính toán hàng nghìn node chỉ trong vài mili-giây.

3. Chuyển dịch không gian: Từ Grid-based sang Navigation Mesh

3.1. Sự bùng nổ tổ hợp của bản đồ lưới (Grid)

Phương pháp sơ khai nhất để máy tính "đọc" bản đồ là chia không gian thành một ma trận lưới (Grid-based). Dù dễ lập trình, nhưng khi áp dụng vào các màn chơi thế giới mở rộng lớn, Grid sinh ra một lượng khổng lồ các node vô ích (ví dụ: một bãi cỏ trống trải cũng bị chia thành hàng vạn node). Điều này dẫn đến độ phức tạp không gian $O(bd)$ (với b là hệ số rẽ nhánh, d là độ sâu), gây tràn bộ nhớ đệm (Cache miss) và làm cạn kiệt RAM.

Bên cạnh đó, lưới sinh ra rác tính toán mang tên **Path Symmetries** (tính dư thừa đường đi) – khi có quá nhiều con đường di chuyển ziczac có cùng một chi phí. Để khắc phục, thuật toán **Jump Point Search (JPS)** được áp dụng để "nhảy" qua các node không có điểm chuyển hướng (forced neighbors), giúp cắt tía các nhánh đối xứng và giảm đến 70% bộ nhớ cần thiết.

3.2. Giải pháp hình học: Navigation Mesh (NavMesh)

Để tối ưu hóa triệt để, thiết kế game hiện đại chuyển sang sử dụng Navigation Mesh. Thay vì chia nhỏ, NavMesh sử dụng các đa giác lồi (Convex Polygons) ghép lại với nhau để bao phủ các bề mặt có thể di chuyển.

Bằng cách này, một căn phòng rộng lớn không còn là một ma trận 100×100 node, mà được gộp lại thành một đa giác lồi duy nhất đại diện cho một node trên đồ thị. Kỹ thuật cắt đa giác (ear-clipping triangulation) và các thuật toán nâng cao như **Polyanya** giúp việc tìm đường trên NavMesh đạt được độ chính xác tuyệt đối mà không cần phải xấp xỉ hóa đồ thị, giải phóng sức mạnh CPU cho các tác vụ đồ họa khác.

4. Thử thách môi trường động và Giải pháp hệ thống

Một màn chơi game không phải là một bức tranh tĩnh. Việc các cửa đóng lại, cây đổ xuống, hay một vật cản xuất hiện bất ngờ đòi hỏi AI phải tính toán lại đường đi (Re-pathing). Nếu gọi lại A* từ đầu cho mọi NPC, CPU sẽ quá tải.

- **Tái sử dụng dữ liệu tính toán:** Các thuật toán như **D* Lite** được thiết kế riêng cho môi trường động. Thay vì tính lại từ đầu, thuật toán này lưu trữ và cập nhật lại trọng số của các cạnh bị thay đổi trên đồ thị cục bộ, giúp tiết kiệm chi phí tính toán.
- **Thiết kế đa luồng (Multi-threading):** Dưới góc độ kiến trúc phần mềm, hệ thống pathfinding sẽ được đẩy ra các luồng xử lý nền (Worker Threads/Thread Pool). Điều này đảm bảo Main Thread (luồng xử lý render đồ họa và nhận input của người chơi) không bao giờ bị block. Khi Worker Thread tìm xong đường đi, nó sẽ trả kết quả về thông qua các hàm Callback hoặc Event Queue.

5. Kết luận

Thuật toán tìm đường trong thiết kế Level không đơn thuần là việc gọi một hàm có sẵn trong Game Engine. Nó là một tổ hợp các bài toán kỹ thuật phức tạp đòi hỏi sự am hiểu sâu sắc về Lý thuyết đồ thị, Cấu trúc dữ liệu và Tối ưu hóa kiến trúc máy tính. Việc áp dụng đúng cấu trúc (từ Min-Heap cho danh sách mở, đến NavMesh cho không gian) chính là chìa khóa để xây dựng các hệ thống trí tuệ nhân tạo mượt mà, chân thực và không tiêu tốn quá nhiều tài nguyên phần cứng.

II. Bảng tổng hợp đánh giá độ tin cậy tài liệu

ST T	Tên tài liệu & Tác giả / Năm xuất bản	Nguồn / Cơ quan lưu trữ	Đánh giá độ tin cậy & Giá trị học thuật (Phản biện phương pháp nghiên cứu)
1	<i>A Navigation Mesh-Based Pathfinding Implementation in CET Designer</i> (Björkman, M., 2021)	DiVA Portal (Hệ thống Lưu trữ Học thuật Thụy Điển)	Mức độ: Rất cao. Bài báo có giá trị thực tiễn lớn khi đối chiếu trực tiếp mã nguồn C++ giữa đồ thị Waypoint và NavMesh. Phương pháp cắt đa giác (ear-clipping) được chứng minh là hiệu quả để tạo không gian di chuyển chính xác, đặc biệt phù hợp để thiết kế bản đồ cho các môi trường hẹp nhiều vật cản như kiến trúc trong nhà (house maps) hay các phòng giải đồ.
2	<i>Compromise-free Pathfinding on a Navigation Mesh</i> (Cui, X. và cộng sự, 2017)	IJCAI (Hội nghị Quốc tế về Trí tuệ Nhân tạo)	Mức độ: Tuyệt đối. Nguồn tài liệu từ hội nghị top đầu thế giới. Thay vì đánh đổi bộ nhớ bằng lưới đồ thị, nhóm nghiên cứu chứng minh bằng giải tích và đại số tuyến tính sự tối ưu của thuật toán Polyanya, cho phép tìm quỹ đạo ngắn nhất xác định trên đa giác lồi mà không bị sai số hệ thống.
3	<i>Applying pathfinding techniques on the development of an Android game</i>	UFJF (Đại học Liên bang Juiz de Fora)	Mức độ: Đáng tin cậy. Phản biện sắc bén về giới hạn phần cứng. Nghiên cứu chỉ ra rằng khi triển khai AI trên môi trường bị giới hạn nghiêm ngặt về RAM và CPU (như thiết bị di động), việc sử dụng các cấu trúc dữ liệu phức tạp sẽ gây rò rỉ bộ nhớ. Tác giả đề xuất

	(Ferreira, R., 2013)		tối giản hàm Heuristic để duy trì flow của trò chơi.
4	<i>Jump Point Search Pathfinding in 4-connected Grids</i> (Harabor, D. và Grastien, A., 2025)	arXiv (Kho lưu trữ số Đại học Cornell)	Mức độ: Tuyệt đối. Nghiên cứu có tính cập nhật cao, giải quyết triệt để bài toán tính dư thừa đường đi (Path Symmetries) của A^* . Bằng cách ép đồ thị về dạng cây và "nhảy" qua các node không cần thiết, thuật toán JPS tiết kiệm tài nguyên tính toán đáng kể trong các bản đồ không gian rộng.
5	<i>Comparison of Grid-based Pathfinding Algorithms in Different Map Types</i> (Lautanen, L., 2020)	LUTPub (Đại học LUT, Phần Lan)	Mức độ: Cao. Sử dụng phương pháp benchmark thực nghiệm. Tác giả đo lường độ phức tạp thời gian (Time Complexity) dựa trên cấu trúc bản đồ (map topologies) và sự phân bố của vật cản. Kết quả đánh giá cung cấp cái nhìn trực quan về sự biến thiên hiệu suất của JPS và A^* .
6	<i>Pathfinding in Computer Games</i> (Millingko, 2011)	Arrow @ TU Dublin (Đại học Công nghệ Dublin)	Mức độ: Đáng tin cậy. Tài liệu mang tính nền tảng chuẩn xác. Phân tích cách Game AI tận dụng cấu trúc dữ liệu tiền tính toán (precomputed data structure) để ra quyết định di chuyển. Bài báo so sánh sâu sắc sự khác biệt về không gian bộ nhớ khi duyệt đồ thị theo chiều sâu (DFS) và chiều rộng (BFS).

7	<i>Review of A* Navigation Mesh Pathfinding as the Alternative of AI for Ghosts Agent...</i> (Pramana, A. và cộng sự, 2020)	Semantic Scholar	Mức độ: Đáng tin cậy. Cung cấp góc nhìn kỹ thuật về lập trình tác tử (Agent). Nghiên cứu trực quan hóa cấu trúc của NavMesh Agent và NavMesh Obstacle, phân tích cách tính toán và chạm động thông qua các vector hướng di chuyển trong không gian game 3D.
8	<i>Comparing Path-finding Algorithms and Machine Learning Model</i> (Ruotsalainen, J., 2022)	Theseus (ĐH Khoa học Ứng dụng Phần Lan)	Mức độ: Cao. Bài báo mang tính phản biện sâu sắc khi đặt thuật toán tìm đường kinh điển lên bàn cân với Reinforcement Learning (Học tăng cường). Phương pháp đánh giá chỉ ra rằng chi phí huấn luyện model AI quá đắt đỏ và thiếu tính ổn định so với thuật toán A* trên các engine game hiện tại.
9	<i>Review of Various A* Pathfinding Implementations in Game Autonomous Agent</i> (Setyawan, A. và cộng sự, 2021)	IJNMT (Tạp chí Công nghệ Truyền thông Mới)	Mức độ: Cao. Benchmark chi tiết trường hợp xấu nhất (worst-case scenario) khi A* thoái hóa. Phân tích chỉ ra sự cần thiết của việc sử dụng Min-Heap (trong Java/C++) cho tập Open List để giữ độ phức tạp thời gian xử lý các node ở mức $O(\log V)$.

10	<i>Multi-threaded Recast-Based A* Pathfinding for Scalable Navigation in Dynamic Game Environments</i> (Wang, Y. và cộng sự, 2026)	arXiv (Kho lưu trữ số Đại học Cornell)	Mức độ: Tuyệt đối. Cập nhật phương pháp xử lý kiến trúc hệ thống hiện đại. Thay vì chạy Pathfinding trên Main Thread gây nghẽn cổ chai (bottleneck), nhóm nghiên cứu tối ưu hóa việc phân luồng (Multi-threading) để tái tính toán đường đi trong môi trường động, duy trì FPS ổn định.
----	--	--	---

III. Danh mục tài liệu tham khảo

Danh mục dưới đây được trình bày theo chuẩn Harvard (Author-Date), sắp xếp theo thứ tự bảng chữ cái (A-Z) dựa trên tên họ của tác giả đầu tiên.

1. Björkman, M., 2021. A Navigation Mesh-Based Pathfinding Implementation in CET Designer. *DiVA Portal*, [Trực tuyến] Có tại: <https://www.diva-portal.org/smash/get/diva2:1560399/FULLTEXT01.pdf> [Truy cập 22 Tháng 3 2026].
2. Cui, X., Harabor, D. và Grastien, A., 2017. Compromise-free Pathfinding on a Navigation Mesh. *IJCAI (International Joint Conferences on Artificial Intelligence)*, tr. 49-55. [Trực tuyến] Có tại: <https://www.ijcai.org/proceedings/2017/0070.pdf> [Truy cập 22 Tháng 3 2026].
3. Ferreira, R., 2013. Applying pathfinding techniques on the development of an Android game. *Repositório Institucional da UFJF*, [Trực tuyến] Có tại: <https://www2.ufjf.br/getcomp/wp-content/uploads/sites/645/2013/03/Applying-pathfinding-techniques-on-the-development-of-an-Android-game.pdf> [Truy cập 22 Tháng 3 2026].
4. Harabor, D. và Grastien, A., 2025. Jump Point Search Pathfinding in 4-connected Grids. *arXiv preprint arXiv:2501.14816*, [Trực tuyến] Có tại: <https://arxiv.org/pdf/2501.14816> [Truy cập 22 Tháng 3 2026].
5. Lautanen, L., 2020. Comparison of Grid-based Pathfinding Algorithms in Different Map Types. *LUTPub*, [Trực tuyến] Có tại: https://lutpub.lut.fi/bitstream/handle/10024/168617/kandidaatintyo_leevi_lautanen.pdf [Truy cập 22 Tháng 3 2026].

6. Millingko, 2011. Pathfinding in Computer Games. *Arrow @ TU Dublin*, [Trực tuyến] Có tại: <https://arrow.tudublin.ie/cgi/viewcontent.cgi?article=1063&context=itbj> [Truy cập 22 Tháng 3 2026].
7. Pramana, A. và cộng sự, 2020. Review of A* Navigation Mesh Pathfinding as the Alternative of Artificial Intelligent for Ghosts Agent on the Pacman Game. *Semantic Scholar*, [Trực tuyến] Có tại: <https://pdfs.semanticscholar.org/d4dc/e2a09288cba5f55b83b508f9c5ed84ed19c2.pdf> [Truy cập 22 Tháng 3 2026].
8. Ruotsalainen, J., 2022. Comparing Path-finding Algorithms and Machine Learning Model. *Theseus*, [Trực tuyến] Có tại: https://www.theseus.fi/bitstream/handle/10024/851091/Ruotsalainen_Joonas.pdf [Truy cập 22 Tháng 3 2026].
9. Setyawan, A., Yulianto, B. và Hartanto, B., 2021. Review of Various A* Pathfinding Implementations in Game Autonomous Agent. *International Journal of New Media Technology*, 8(1). [Trực tuyến] Có tại: <https://ejournals.umn.ac.id/index.php/IJNMT/article/download/1075/799> [Truy cập 22 Tháng 3 2026].
10. Wang, Y. và cộng sự, 2026. Multi-threaded Recast-Based A* Pathfinding for Scalable Navigation in Dynamic Game Environments. *arXiv preprint arXiv:2602.04130*, [Trực tuyến] Có tại: <https://arxiv.org/pdf/2602.04130> [Truy cập 22 Tháng 3 2026].